REPORTE TAREA 2 y 3
ALGORITMOS Y COMPLEJIDAD

«Encontrando Diferencias entre dos Secuencias»

Vicente Vizcarra Rubio

19 de junio de 2025

01:28

Resumen

El presente informe aborda el problema de encontrar diferencias entre dos cadenas de texto, a partir de la subsecuencia común más larga (LCS, por sus siglas en inglés), con el objetivo de comparar el rendimiento de dos paradigmas algorítmicos: una solución por fuerza bruta y una basada en programación dinámica. Se desarrollaron ambas implementaciones considerando una entrada estándar con múltiples casos de prueba. Para evaluar el rendimiento de los algoritmos se generó un conjunto de datos, con 10 casos de prueba diseñados para representar distintos tamaños de entrada, lo que permitió analizar el tiempo de ejecución y uso estimado de memoria en función del tamaño de las cadenas comparadas. Los resultados experimentales muestran una clara superioridad de la programación dinámica frente a la fuerza bruta a medida que el tamaño crece, validando así su idoneidad para este tipo de problemas.

Índice

1. Introducción 2

2. Diseño y Análisis de Algoritmos 3

3. Implementaciones 6

4. Experimentos 7

5. Conclusiones 11

6. Condiciones de entrega 12

A. Apéndice 1 13

1. Introducción

El análisis y diseño de algoritmos es un área fundamental en Ciencias de la Computación, debido a que permite evaluar la eficiencia de diferentes métodos para resolver problemas de carácter computacional. El estudio de la complejidad temporal y espacial de los algoritmos es esencial para poder elegir la solución más adecuada teniendo en cuenta los recursos disponibles y el tamaño de los datos de entrada. Dentro de este contexto, uno de los problemas clásicos es el de encontrar las diferencias mínimas entre dos cadenas de texto, lo cual puede abordarse a través del cálculo de la *Subsecuencia Común Más Larga* (LCS).

El cálculo del LCS permite identificar qué elementos se mantienen entre dos secuencias, y a partir de ello, inferir las partes que difieren. Este problema es interesante, ya que existen múltiples estrategias para resolverlo, con distintas complejidades temporales y espaciales. A nivel teórico, el enfoque de fuerza bruta proporciona una visión clara del espacio de soluciones posibles, mientras que la programación dinámica permite reducir considerablemente el tiempo de ejecución mediante el uso de subproblemas intermedios.

En este informe, se propone comparar ambos paradigmas mediante una implementación modular y su posterior evaluación experimental. El objetivo es observar el comportamiento práctico de cada algoritmo bajo un conjunto de entradas con longitudes variadas, buscando evidenciar cómo la complejidad teórica se manifiesta en la práctica.

2. Diseño y Análisis de Algoritmos

En esta sección se presentan dos soluciones al problema de encontrar las diferencias mínimas entre dos secuencias, siguiendo dos paradigmas de diseño de algoritmos vistos en el curso: fuerza bruta y programación dinámica. Ambas soluciones buscan determinar las posiciones donde las cadenas difieren, basándose en la subsecuencia común más larga (LCS) como estructura intermedia.

A continuación, se describe cada algoritmo con su pseudocódigo respectivo, junto a un análisis de complejidad temporal y espacial en función de las longitudes de entrada $n = |s|$ y $m = |t|$.

2.1. Fuerza Bruta

> *"Indeed, brute force is a perfectly good technique in many cases; the real question is, can we use brute force in such a way that we avoid the worst-case behavior?"*
>
> — Knuth, 1998 [2]

El enfoque de fuerza bruta consiste en generar de forma recursiva todas las posibles subsecuencias comunes entre las cadenas s y t, seleccionando aquella de mayor longitud. A partir de esta subsecuencia común más larga (LCS), se reconstruyen las diferencias entre las cadenas comparando los bloques que quedan entre cada carácter coincidente.

El procedimiento recorre todas las combinaciones posibles avanzando en una de las dos cadenas o en ambas, y acumulando la subsecuencia cuando los caracteres coinciden. La diferencia entre las cadenas se obtiene después recorriendo s y t en paralelo, detectando fragmentos que no coinciden con la LCS.

Este método cumple con la definición clásica de fuerza bruta, ya que no evita ningún subproblema ni aplica poda o memoria intermedia.

Algoritmo 1: Generación de LCS por fuerza bruta y reconstrucción de diferencias

```
1  Procedure BRUTELCS(s, t, i, j, actual)
2      if i = |s| or j = |t| then
3          if |actual| > |mejor_subsecuencia| then
4              mejor_subsecuencia ← actual
5          return
6      if s[i] = t[j] then
7          BRUTELCS(s, t, i + 1, j + 1, actual + s[i])
8      BRUTELCS(s, t, i + 1, j, actual)
9      BRUTELCS(s, t, i, j + 1, actual)
10 Procedure GENERARDIFERENCIAS(s, t)
11     mejor_subsecuencia ← ""
12     BRUTELCS(s, t, 0, 0, "")
13     lcs ← mejor_subsecuencia
14     Comparar s y t usando lcs para extraer diferencias
```

Complejidad temporal: En el peor de los casos, se generan todas las subsecuencias de s y se comparan con todas las de t. Dado que una cadena de largo n tiene 2^n subsecuencias, la complejidad de este algoritmo es $O(3^{\max(n,m)})$ al recorrer el espacio recursivo de decisiones.

Complejidad espacial: El algoritmo utiliza $O(n+m)$ espacio para representar la cadena actual acumulada y para la mejor subsecuencia encontrada. La pila de recursión tiene una profundidad máxima de $O(n+m)$.

Este enfoque no es viable para entradas grandes, pero cumple su propósito como referencia para análisis comparativo, y como demostración del paradigma de fuerza bruta.

2.2. Programación Dinámica

Dynamic programming is not about filling in tables. It's about smart recursion!

Erickson, 2019 [1]

2.2.1. Descripción de la solución recursiva

El algoritmo se basa en calcular la subsecuencia común más larga (LCS) entre las cadenas s y t, utilizando programación dinámica. Esta subsecuencia permite identificar los fragmentos compartidos y, por complemento, obtener las diferencias entre ambas cadenas. La solución recursiva compara los caracteres desde el final hacia el inicio, acumulando la longitud del LCS cuando los caracteres coinciden y eligiendo la mejor opción cuando no lo hacen.

2.2.2. Relación de recurrencia

Sea $dp[i][j]$ la longitud de la LCS entre los prefijos $s[0..i-1]$ y $t[0..j-1]$, entonces:

$$dp[i][j] = \begin{cases} 0 & \text{si } i = 0 \text{ o } j = 0 \\ dp[i-1][j-1]+1 & \text{si } s[i-1] = t[j-1] \\ \max(dp[i-1][j], dp[i][j-1]) & \text{en otro caso} \end{cases}$$

2.2.3. Identificación de subproblemas

Los subproblemas corresponden a todos los pares de prefijos $(s[0..i], t[0..j])$ para $0 \le i \le n$ y $0 \le j \le m$. El total de subproblemas es $O(n \cdot m)$.

2.2.4. Estructura de datos y orden de cálculo

Se utiliza una matriz bidimensional dp de tamaño $(n+1)\times(m+1)$, donde $n = |s|$ y $m = |t|$. La matriz se llena en orden ascendente sobre i y j, es decir, fila por fila o columna por columna. Una vez llena la tabla, se reconstruye la LCS desde el final hacia el inicio, y se usan sus posiciones para extraer las diferencias entre bloques consecutivos de s y t.

2.2.5. Algoritmo utilizando programación dinámica

Algoritmo 2: Cálculo de LCS y diferencias usando programación dinámica

```
Procedure DPDifferences(s, t)
    n ← |s|, m ← |t|
    Crear matriz dp[0..n][0..m] inicializada en cero
    for i = 1 to n do
        for j = 1 to m do
            if s[i − 1] = t[j − 1] then
                dp[i][j] ← dp[i − 1][j − 1] + 1
            else
                dp[i][j] ← máx(dp[i − 1][j], dp[i][j − 1])
    lcs ← BuildLCS(dp, s, t)
    Comparar s y t usando lcs para extraer diferencias
Procedure BuildLCS(dp, s, t)
    i ← |s|, j ← |t|, res ← lista vacía
    while i > 0 y j > 0 do
        if s[i − 1] = t[j − 1] then
            Agregar (i − 1, j − 1) al inicio de res
            i ← i − 1, j ← j − 1
        else if dp[i − 1][j] ≥ dp[i][j − 1] then
            i ← i − 1
        else
            j ← j − 1
    return res
```

Complejidad temporal: El algoritmo tiene complejidad $O(n \cdot m)$ debido al llenado de la matriz y la posterior reconstrucción de la LCS.

Complejidad espacial: Se utiliza $O(n \cdot m)$ de espacio para la matriz de programación dinámica. En algunos casos, puede optimizarse a $O(\text{mín}(n, m))$ si solo interesa la longitud del LCS.

Este enfoque es eficiente y escalable, lo que lo hace adecuado para entradas grandes en tiempo razonable.

3. Implementaciones

Esta tarea fue desarrollada en el lenguaje C++. Cada paradigma algorítmico se encuentra separado en directorios distintos, facilitando su mantenimiento y evaluación independiente.

El algoritmo de fuerza bruta se encuentra en la carpeta code/brute_force. El archivo principal es brute_force.cpp, el cual lee los datos de entrada y procesa cada caso utilizando la función obtener_diferencias, definida en algorithm/sequence_difference.cpp. Esta función implementa una estrategia recursiva para generar todas las subsecuencias comunes posibles, seleccionando la más larga.

El algoritmo basado en programación dinámica se ubica en code/dynamic_programming, con su archivo principal dynamic_programming.cpp. Este programa también llama a obtener_diferencias desde el subdirectorio algorithm, donde se construye la tabla de LCS y se reconstruye la diferencia entre las cadenas.

Ambos programas utilizan entradas desde archivos de texto ubicados en sus respectivas carpetas data/, y escriben la salida procesada y las mediciones en archivos separados. Cada uno cuenta con su propio makefile para compilar y ejecutar los casos de prueba automáticamente.

El código fuente completo, junto con los scripts de generación de entrada y graficación de resultados, se encuentra disponible en el siguiente repositorio privado:

https://github.com/VicenteVR/INF221-2025-1-TAREA-2-3

4. Experimentos

> "*Non-reproducible single occurrences are of no significance to science.*"
>
> —Popper,2005 [3]

Los experimentos realizados en esta tarea se llevaron a cabo en un entorno controlado para garantizar la reproducibilidad de los resultados. La infraestructura utilizada es la siguiente:

- **Hardware:** Procesador 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz 2.42 GHz, 8 GB RAM, almacenamiento SSD Intel Optane.
- **Software:** Sistema operativo Ubuntu 22.04 LTS corriendo bajo WSL (Windows Subsystem for Linux) sobre Windows 10. Compilador `g++ 11.4.0` con soporte para `-std=c++17`. Scripts complementarios en Python 3.10.12.

Ambos algoritmos fueron compilados utilizando un `makefile`, lo que permite replicar el proceso sin ambigüedades. Las mediciones se realizaron ejecutando ambos programas sobre un conjunto de datos fijo, generado por un script controlado, que produce un archivo con 10 casos de prueba variados en tamaño, estructura y contenido. Cada ejecución procesa exactamente los mismos datos de entrada, asegurando una comparación justa entre los paradigmas.

Se midió el tiempo de ejecución de cada caso utilizando la librería estándar `<chrono>` en C++, y se estimó el uso de memoria con base en las estructuras activas utilizadas por cada algoritmo. Los resultados se almacenaron en archivos de texto para su posterior graficación.

El propósito de los experimentos es comparar de forma empírica la eficiencia y escalabilidad de dos estrategias algorítmicas para resolver el mismo problema, en condiciones equivalentes y sobre entradas diseñadas para mostrar diferencias significativas de rendimiento.

4.1. Dataset (casos de prueba)

El conjunto de casos de prueba utilizado para esta tarea fue generado con el objetivo de analizar el comportamiento práctico de los algoritmos en entradas variadas y controladas. Se definieron $K = 10$ casos, con pares de cadenas de texto (s, t) que varían en tamaño total desde 10 hasta 320 caracteres. Los strings fueron diseñados para asegurar una mezcla de patrones repetitivos, diferencias parciales y presencia de espacios, lo cual permite evaluar distintos tipos de dificultad para los algoritmos.

Los tamaños fueron elegidos de forma incremental, comenzando con secuencias cortas ($|s| = |t| = 5$) y aumentando progresivamente hasta cadenas de tamaño 160. Para cada par, se mantuvo una cierta similitud estructural entre s y t, permitiendo que el algoritmo encuentre subsecuencias comunes sin ser triviales. Algunos ejemplos incluyen cadenas con diferencias pequeñas, diferencias múltiples, cadenas muy similares y otras más divergentes.

- Casos 1-2: cadenas cortas con diferencias simples.

- Casos 3-4: cadenas con espacios y bloques modificados parcialmente.
- Casos 5-10: cadenas largas con patrones estructurados, diseñadas para poner a prueba la escalabilidad y eficiencia de los algoritmos.

Los casos fueron generados mediante un script en Python que garantiza la reproducibilidad de los experimentos. Este script fue ejecutado una sola vez para producir un archivo `a.txt`, que luego se utilizó como entrada para ambos algoritmos. Así se asegura una comparación justa.

Debido a la naturaleza exponencial del enfoque por fuerza bruta, este solo fue ejecutado hasta el segundo caso del conjunto. A partir del tercer caso ($|s| = |t| = 20$), el tiempo de ejecución superó las dos horas sin finalizar, por lo que se omitieron esos resultados. En cambio, el enfoque de programación dinámica pudo ejecutarse completamente en todos los casos, demostrando su superioridad en términos de escalabilidad.

Este dataset, al ser estático y variado, permite un análisis claro del rendimiento de ambos enfoques y refleja adecuadamente la motivación de la tarea: contrastar paradigmas algorítmicos bajo condiciones equivalentes.

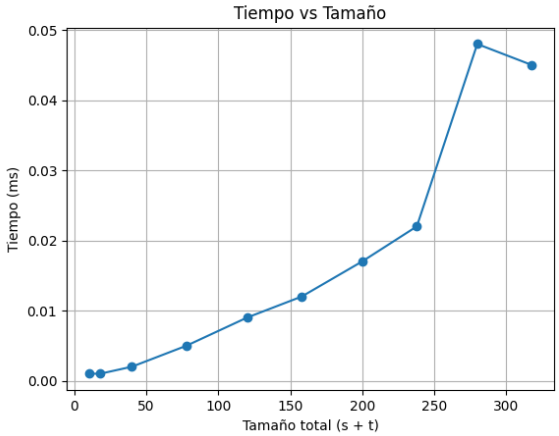
4.2. Resultados

Los resultados experimentales se obtuvieron ejecutando ambos algoritmos sobre un conjunto fijo de 10 casos de prueba, previamente generados por un script de Python ubicado en `code/dynamic_programming/scripts/`. Cada caso contiene dos cadenas de texto (s, t) con longitudes iguales, variando desde 5 hasta 160 caracteres, con diferencias estructuradas entre ellas.

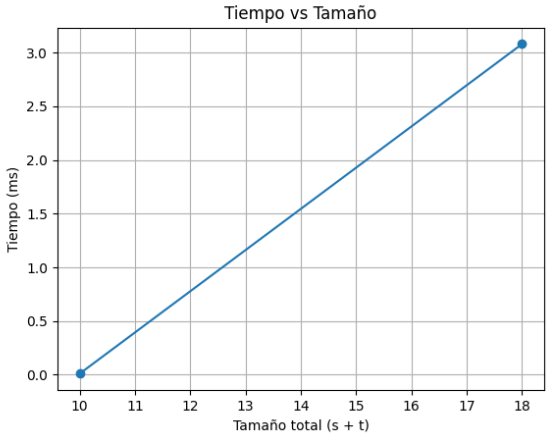
Durante la ejecución, ambos programas escribieron las mediciones de tiempo (en milisegundos) y una estimación del uso de memoria (en KB) en archivos de texto ubicados en `data/measurements/a.txt`, los cuales pueden ser regenerados en cualquier entorno compatible simplemente ejecutando:

```
make              # desde code/brute_force o code/dynamic_programming
```

Las gráficas fueron generadas a partir de estos archivos usando el script `plot_generator.py`. El script lee las mediciones y construye dos gráficos: uno de tiempo de ejecución y otro de uso de memoria, guardándolos en `data/plots/`. Las figuras resultantes fueron exportadas como archivos PNG y agregadas al directorio `report/images/` para su inclusión en el informe.

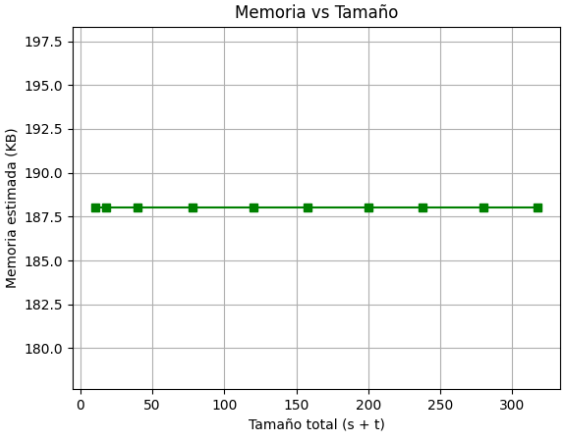


(a) Tiempo vs Tamaño (DP)

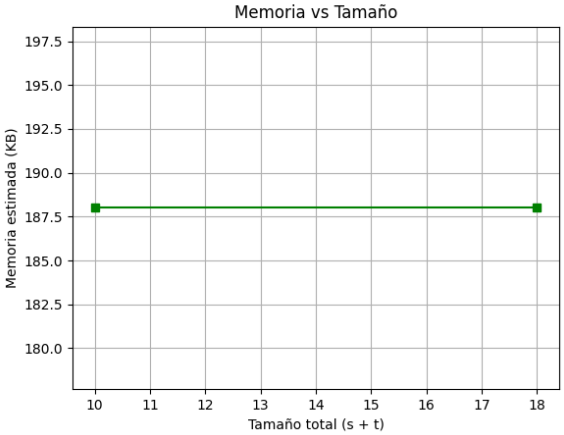


(b) Tiempo vs Tamaño (Fuerza Bruta)

Figura 1: Tiempo de ejecución para ambos algoritmos



(c) Memoria vs Tamaño (DP)



(d) Memoria vs Tamaño (Fuerza Bruta)

Figura 2: Uso de memoria estimado para ambos algoritmos

En la Figura 1, se aprecia que el algoritmo de programación dinámica escala de forma eficiente y estable, con tiempos de ejecución que se mantienen por debajo de 0.05 ms incluso para entradas grandes. En contraste, la fuerza bruta solo logra ejecutarse para los dos primeros casos, presentando un salto abrupto de tiempo entre entradas pequeñas, lo cual refleja su crecimiento exponencial.

La Figura 2 muestra que ambos algoritmos presentan un uso de memoria estimado constante, dado que se usó una estimación simplificada basada en estructuras de datos principales. Sin embargo, en la práctica, fuerza bruta requeriría mayor uso de pila por recursión profunda, lo cual podría generar errores de ejecución en casos grandes.

En conjunto, los resultados experimentales muestran que la programación dinámica no solo es teóricamente más eficiente, sino que también se comporta de forma significativamente mejor en la práctica,

tanto en tiempo como en escalabilidad.

5. Conclusiones

Los resultados obtenidos en esta tarea nos permiten concluir que la programación dinámica no solo es teóricamente más eficiente que la fuerza bruta, sino que también se manifiesta como la opción superior al momento de resolver instancias prácticas del problema de diferencias entre secuencias. La incapacidad de la fuerza bruta para escalar más allá de entradas pequeñas valida las predicciones sobre su complejidad exponencial, mientras que la solución dinámica mantiene tiempos de ejecución bajos y consistentes incluso en los casos más grandes evaluados.

Estos hallazgos reflejan la importancia de aplicar paradigmas algorítmicos adecuados según la naturaleza del problema, y evidencian cómo una estrategia permite abordar situaciones que serían intratables de otro modo. El contraste entre ambos enfoques también refuerza el valor del análisis experimental como herramienta complementaria al análisis teórico, especialmente cuando se desea observar el comportamiento real de algoritmos en condiciones controladas.

En definitiva, el experimento confirma que el paradigma de programación dinámica ofrece una solución viable, eficiente y reproducible para comparar secuencias mediante la subsecuencia común más larga, y que elegir el enfoque correcto marca una diferencia tanto en rendimiento como en aplicabilidad.

6. Condiciones de entrega

- La tarea se realizará individualmente (esto es grupos de una persona), sin excepciones.
- La entrega debe realizarse vía `http://aula.usm.cl` en un tarball en el área designada al efecto, en el formato `tarea-2 y 3-rol.tar.gz` (`rol` con dígito verificador y sin guión).

  Dicho tarball debe contener las fuentes en LaTeX 2ε (al menos `tarea-2 y 3.tex`) de la parte escrita de su entrega, además de un archivo `tarea-2 y 3.pdf`, correspondiente a la compilación de esas fuentes.
- Si se utiliza algún código, idea, o contenido extraído de otra fuente, este **debe** ser citado en el lugar exacto donde se utilice, en lugar de mencionarlo al final del informe.
- Asegúrese que todas sus entregas tengan sus datos completos: número de la tarea, ramo, semestre, nombre y rol. Puede incluirlas como comentarios en sus fuentes LaTeX (en TeX comentarios son desde % hasta el final de la línea) o en posibles programas. Anótese como autor de los textos.
- Si usa material adicional al discutido en clases, detállelo. Agregue información suficiente para ubicar ese material (en caso de no tratarse de discusiones con compañeros de curso u otras personas).
- No modifique `preamble.tex`, `tarea_main.tex`, `condiciones.tex`, estructura de directorios, nombres de archivos, configuración del documento, etc. Sólo agregue texto, imágenes, tablas, código, etc. En el códigos funte de su informe, no agregue paquetes, ni archivos .tex (a excepción de que agregue archivos en `/tikz`, donde puede agregar archivos .tex con las fuentes de gráficos en `TikZ`).
- La fecha límite de entrega es el día 6 de junio de 2025.

NO SE ACEPTARÁN TAREAS FUERA DE PLAZO.

- Nos reservamos el derecho de llamar a interrogación sobre algunas de las tareas entregadas. En tal caso, la nota de la tarea será la obtenida en la interrogación.

NO PRESENTARSE A UN LLAMADO A INTERROGACIÓN SIN JUSTIFICACIÓN PREVIA SIGNIFICA AUTOMÁTICAMENTE NOTA 0.

A. Apéndice 1

Aquí puede agregar tablas, figuras u otro material que no se incluyó en el cuerpo principal del documento, ya que no constituyen elementos centrales de la tarea. Si desea agregar material adicional que apoye o complemente el análisis realizado, puede hacerlo en esta sección.

> Esta sección es solo para material adicional. El contenido aquí no será evaluado directamente, pero puede ser útil si incluye material que será referenciado en el cuerpo del documento. Por lo tanto, asegúrese de que cualquier elemento incluido esté correctamente referenciado y justificado en el informe principal.

Referencias

[1] Jeff Erickson. *Algorithms*. Jun. de 2019. ISBN: 978-1-792-64483-2.

[2] Donald E. Knuth. *The art of computer programming, volume 3: (2nd ed.) sorting and searching*. USA: Addison Wesley Longman Publishing Co., Inc., 1998. ISBN: 0201896850.

[3] K. Popper. *The Logic of Scientific Discovery*. Routledge Classics. Taylor & Francis, 2005. ISBN: 9781134470020. URL: https://books.google.cl/books?id=LWSBAgAAQBAJ.